TamilEOT: A Dataset and Model for Semantic End-of-Turn Detection in Tamil Telephone Speech

Santhoshkumar V
santhoshdon000555@gmail.com
https://github.com/santhosh-005/tamil-eot

Abstract

A voice agent has to decide, at every pause, whether the user has finished speaking. Without a model of the language that decision falls back to a fixed silence timeout: set it short and the agent interrupts, set it long and every turn pays the full wait. Open semantic end-of-turn detectors exist, but to our knowledge none covers a South Indian language. We release TamilEOT: 18,485 labelled turn boundaries cut from 116 real Tamil telephone conversations, and two audio-only detectors fine-tuned from Smart Turn v3. On a held-out split of 4,168 clips from 30 unseen calls, accuracy rises from 70.30% zero-shot to 83.71% (8.7 MB, 83 ms on one CPU thread) and 86.13% (21 MB, 143 ms); ROC-AUC rises 0.751 → 0.921. We also report what building it cost. Rule-derived labels, checked against a blind human listening pass, were right 95.9% of the time on the positive class and 44.4% on the negative class — below chance — because the rule answered a different question than the model is asked. Replacing them with an audio-LLM labeller measured at 97.5% human agreement cost US$5.69. Of every training lever we measured, only encoder capacity moved the result; three runs at identical config and seed span 0.87 accuracy points, which is the floor below which none of our other deltas mean anything. Replaying the same labelled boundaries through the production VAD and streaming adapter costs a further 2.60 points, and 7.8% of boundaries are never surfaced to the model at all. Data, weights, code and every negative result are public.

1 Introduction

Turn-taking is the part of a voice agent users notice first when it is wrong. At each pause the agent must answer one question: has this person finished, or are they thinking? Voice activity detection answers a different question — *is there sound* — so systems built on VAD alone add a fixed silence timeout on top. That timeout is a single dial with two bad ends. Short, and the agent talks over a speaker who paused mid-sentence. Long, and every completed turn waits for a timer that exists only to catch the pauses.

A semantic end-of-turn (EOT) detector removes the dial. It listens to the last few seconds of the speaker's audio and predicts completion from prosody, without waiting for a transcript. Open models of this kind exist — Smart Turn v3 [7] is the most widely deployed — but their released weights cover a fixed language list, and Tamil is not on it. Hosted speech APIs offer Tamil turn detection, but not as something you can inspect, retrain, or run locally. For Tamil there was no open dataset to train on either, which is the actual blocker: the architecture is small and the training script is public.

This paper describes what it took to close that gap for one language, and what we got wrong on the way.

Contributions.

1. **A dataset.** 18,485 labelled turn boundaries from 116 real Tamil narrowband telephone conversations, split by call, released CC BY 4.0 (Section 3). To our knowledge it is the first open dataset for semantic end-of-turn detection in Tamil, and the first in any South Indian language.
2. **A labelling protocol that is validated rather than assumed.** We measured our own rule-derived labels against a blind human pass, found the negative class below chance, diagnosed why, and replaced it with an audio-LLM labeller chosen by measurement. We also report the resulting label-noise ceiling (Section 4).
3. **Two models and a set of negative results.** 70.30% $\rightarrow$ 86.13% on the held-out split. Of six training levers, one worked. We publish the five that did not, together with the run-to-run spread that makes small claims unfalsifiable (Section 6).
4. **A measurement of what serving costs.** Offline accuracy scores a clip cut by the dataset builder; production scores a window cut by a VAD. We replayed the same labelled boundaries through the real VAD and the real streaming adapter and measured the difference (Section 7).

A note on conventions. Two false-positive rates are in circulation and they differ by roughly $3\times$. Smart Turn publishes FP/N, where N is the whole evaluation set, so that FP/N and FN/N sum to the error rate. The standard FPR $=$ FP/(FP + TN) is computed over negatives only. On our test set 10.1% in the first convention is 27.4% in the second. Every table below states which one it uses. The positive class throughout is `complete`, so a false positive is *the agent talking over the user* — the expensive error.

2 Related work

Turn-taking models. Voice Activity Projection [2] learns turn-taking events self-supervised from the future voice activity of both speakers, and needs both channels. Smart Turn v3 [7] takes the deployment-shaped version of the problem: one channel, the user's, and a binary verdict on the last 8 seconds. It is a Whisper-tiny encoder [4] with a classification head, about 8M parameters, quantised to int8 and running in tens of milliseconds on CPU. Weights, data and training code are all open, which is why we fine-tuned it rather than starting over. Its released v3 weights cover 23 languages; the three Indic ones (Bengali, Hindi, Marathi) are all Indo-Aryan, and no South Indian language is included.

Datasets. The ETD dataset [1] is the first public corpus aimed directly at end-turn detection, combining TTS-generated dialogue with real conversational audio. Smart Turn's own training corpus is largely TTS. Neither covers Tamil, and neither is narrowband telephone speech, which is the acoustic condition most Indic voice agents actually run in.

Language-specific work. The closest analogue to this paper is a recent study of Thai end-of-turn detection [8], which builds a Thai baseline from transcribed subtitles and classifies at token boundaries. That work is text-only and depends on a transcript; ours is audio-only and runs before one exists. The two approaches are complementary — a transcript-based detector cannot fire until ASR has emitted, and an audio-based one cannot read lexical cues.

3 Dataset

3.1 Source, and why this corpus

The audio is the `ta_IN_*_{Left,Right}` family of SPRING-INX Tamil R1 [5], released CC BY 4.0 by SPRING Lab, IIT Madras. It is 116 two-party Tamil telephone conversations, and it has one property that decided the whole project: **each conversation ships as one audio file per speaker**, each leg transcribed separately by a human.

Which file the audio came from *is* the speaker label. There is no diarization step and therefore no speaker-error rate propagating into the labels. We verified 116/116 pairs complete with 0 ms duration mismatch between legs. Every clip is cut from one leg only, which is also what a deployed detector receives: a voice agent sees the inbound user stream, not a mixdown.

3.2 Boundaries come from the VAD, not the transcript

The obvious way to find turn boundaries is to use the transcript segment edges. It is wrong here. Summed across both legs, the shipped segments cover about 107% of the call wall clock, which is only possible if they carry lead-in and trailing silence. Their edges therefore inflate apparent overlap and misplace every gap.

We run Silero VAD [6] on each leg independently and take speech onsets and offsets from it. The transcript is used for exactly two things: confirming that a VAD span is real speech on *that* leg (a span with no transcript over it is crosstalk bleed from the other leg), and supplying text for the backchannel filter. Words are apportioned across VAD speech seconds rather than elapsed time — the median segment is 21% silence and the 10th percentile is 53%, so linear interpolation would hand about a fifth of the words to intervals where nobody spoke.

From 116 calls this yields 50,532 candidate boundaries: 20,013 floor changes and 30,519 same-speaker holds.

3.3 Clip geometry

Each sample is the 8 seconds of that speaker's audio ending 200 ms after the speech offset. **Both classes get exactly the same 200 ms of trailing audio.** This matters more than it looks: if positives ended with more silence than negatives, a model could learn to read silence length instead of speech, score well offline, and collapse against a production endpointer with different timing.

We test for that rather than assert it. `09_verify.py` fits a deliberately weak classifier on ten crude global features — energy, duration, spectral tilt, voiced fraction — that carry no prosodic structure. Near-chance performance is the pass condition. Results are in Section 4.4. On the most recent harvest, measured peak amplitude in the final 200 ms is median 0.0021 and p95 0.0225: the trailing window is silence, and its length carries no label information.

3.4 Composition and splits

Boundaries fall into four provenance classes, by how they were detected and whether the two available witnesses — the other speaker's behaviour and the transcriber's segmentation — agreed (Table 1). Provenance is recorded on every row but is *not* a feature: it is unavailable at inference.

Two of the four classes (`change_midseg`, `hold_inter`) were initially held back precisely because the witnesses disagreed and a rule had to break the tie alone. Once a validated third witness existed (Section 4) they were released into the set. A second pass over rejected boundaries with two gates relaxed added a further 2,272 rows.

Split by call, never by clip. Both SPRING-INX releases ship utterance-level splits that place the same recording on both sides — in the R2 release, 511 of 545 recordings appear in train and eval

provenance	detected as	n
`hold_intra`	negative	10,402
`change`	positive	4,660
`hold_inter`	negative	2,343
`change_midseg`	positive	1,080
total		18,485

Table 1: Provenance of the 18,485 boundaries. The shipped label is the labeller's verdict, which overrules provenance on 53% of `hold_intra`.

split	calls	clips	compl.	incompl.
train	71	11,992	7,908	4,084
dev	15	2,325	1,532	793
test	30	**4,168**	2,629	1,539
total	116	18,485	12,069	6,416

Table 2: Splits, assigned *by call*. The test split has not moved since it was first cut.

alike. A turn detector trained across such a split memorises voices instead of learning prosody. We split at the call level (Table 2), so no voice in the test set was heard during training.

4 Labels

4.1 We measured our own labels, and they failed

The first label set was rule-derived: a floor change at a segment end is `complete`; a pause wholly inside one transcript segment is `incomplete`. This is the standard construction, and it is cheap enough that it is rarely questioned.

We ran a blind listening pass: 197 clips stratified by (provenance, label), no label shown, no metadata, audio only, one listen each.

class	grounded in	agreement with listener
`change` (positive)	the other person took the floor	**95.9%**
`hold_intra` (negative)	a pause fell inside a segment	**44.4%**
overall		70.1%

44.4% is below chance. The negative class was worse than a coin flip.

4.2 Why: the rule answered a different question

The positive class works because it is grounded in behaviour that was *recorded in the call*. The other speaker heard the turn end and took the floor. That is a witness.

The negative class had no witness. "A pause fell inside a transcript segment" is not a judgement about completeness — it is a judgement about where a human transcriber pressed enter.

Underneath this sits a genuine ambiguity, and naming it is the useful part. Two defensible questions diverge on a speaker who finishes one sentence and starts another:

	question	who asks it
Q1	is this utterance *syntactically finished*?	our listeners, our prompt, Smart Turn's data
Q2	does the speaker *intend to continue*?	a pause oracle; transcript-derived labels

Our rule-derived negatives answered Q2. Pause-derived labelling of real conversation is a reasonable construction and prior work uses it [1]; it simply is not the question a Smart Turn fine-tune is being trained on. Rewording our evaluation to Q2 was ruled out for the same reason: mixing a Q2 corpus into a Q1-pretrained model teaches two different questions.

We also confirmed that no amount of feature engineering rescues it. Every structural, textual and metadata feature the pipeline computes, combined, predicts the human verdict at AUC 0.637.

labeller	agreement	on `hold_intra`	est. cost, full pool
`gemini-3.7-flash`	**97.5%**	**97.0%**	**$5.77**
`gemini-3.1-pro-preview`	96.4%	93.9%	$41.17
`gemini-3.6-flash`	96.4%	97.0%	$9.10
`gemini-3.5-flash`	93.9%	92.9%	$11.82
`gemini-3-flash-preview`	93.4%	90.9%	$17.09
`gemini-2.5-flash`	91.8%	90.8%	$13.50
`gemini-3.5-flash-lite`	66.5%	68.7%	$1.18
the rule-derived labels	70.1%	44.4%	—

Table 3: Labeller bake-off on the 197 blind-listened clips. Six of seven clear the 90% bar. By McNemar the top three are statistically indistinguishable ($p = 0.75,\ 0.73$), so the tie was broken on price, not on the point estimate.

4.3 Relabelling with an audio LLM

The replacement had to *listen*. We evaluated seven audio-capable LLMs on the same 197 human-labelled clips, with the same prompt, audio only, no transcript and no prior label, run paired through a batch API (Table 3).

`gemini-3.7-flash` agrees with the human listener 97.5% of the time, and 97.0% on the exact class the rules got wrong. It relabelled the full pool for **US$5.69** (a further US$0.78 for the 2,272 rows added later).

Two independent measurements of the same error agree: **53% of the negative class flipped** under relabelling, against **56%** in the human pass. Class balance on the original pool moved from 4,751/11,462 to 10,493/5,720 complete/incomplete; the released set is 12,069/6,416.

Every row retains `label_pipeline`, `llm_verdict` and a `dispute` flag, so the set can be re-derived under a different policy without re-running anything.

4.4 The confound this removed

Relabelling also closed the one structural defect we knew about. `hold_intra` required a pause *inside* a segment, so the negative class was drawn from longer utterances by construction — and that leaked into the audio as voiced fraction. The gate was deciding the label, so the gate's bias rode along with it.

labels	shortcut-probe AUC	`prev_dur` d	`voiced_frac` d
rule-derived	0.601	−0.33	−0.46
relabelled	0.622	**−0.09**	**−0.17**

Both rows are the matched core subsets, so the comparison is like for like; on the full released set `prev_dur` $d = -0.11$. Cohen's d on both leaked features drops to near zero. The probe's AUC goes *up* slightly, $0.601 \rightarrow 0.622$, which we report rather than hide; both sit inside the near-chance band, and the class balance moved at the same time, so AUC is the comparable quantity and accuracy is not.

4.5 The label-noise ceiling

Because both witnesses are recorded per row, the residual label noise can be estimated directly from the QA sample.

subset	share of test	label accuracy
both witnesses agree	62.3%	99.3%
disputed	37.7%	93.5%
weighted		**97.1%**

Above 97.1% a model is fitting labeller error. Our best model is at 86.13%, so roughly 11 points of headroom remain and label quality is not what currently limits it.

Finally, the 197 human labels are themselves one listen to one isolated 8-second clip with no future audio, and are not truth. Three were wrong: all seven labellers contradicted them in the same direction, and on re-listening the labellers were right. Corrected verdicts live in a file that every downstream number is re-derived from, rather than being hardcoded.

5 Model and training

The architecture is unchanged from Smart Turn v3: a Whisper encoder [4] over an 8-second log-mel window, attention pooling, and a binary classification head. We fine-tune the whole stack.

encoder	`openai/whisper-tiny` (8.0M) or `openai/whisper-base` (20.3M)
head	attention pooling → linear binary classifier
loss	`BCEWithLogitsLoss` with per-batch `pos_weight`
schedule	6 epochs, lr 5×10^{-5}, batch 32, seed 0
selection	best epoch on dev accuracy (epoch 3 or 4 in every run)
export	`.pt` → fp32 ONNX → int8 dynamic, on CPU
hardware	one NVIDIA T4 on Google Colab

There is no multi-GPU step anywhere in this work. Both released models come from one notebook and the encoder string is the only difference between them.

Are the released files the models we trained? These are two different claims and we checked both. First, both released ONNX files were re-scored locally on the sealed test set and reproduce their training numbers. Second, re-exporting the released `base` checkpoint produces a graph **bit-identical** to the published one ($\max|\Delta| = 0.00\text{e}{+00}$). Every number in this paper therefore comes from the artefact a reader can download, not from a training log.

6 Results

6.1 The reproducibility floor, first

Before any delta: three `base` runs at identical configuration and identical seed scored **86.23%** / **85.63%** / **85.36%** — a spread of **0.87 points**.

`cudnn.deterministic` constrains only cuDNN ops. The encoder is attention, routed through SDPA, whose flash and memory-efficient backward kernels use atomics; non-associative float addition then produces divergent trajectories over ∼2,244 optimiser steps. Checkpoint selection amplifies it, since different runs peak at different epochs of a noisy curve.

We therefore treat anything below about 1 point as noise, and we ask readers to do the same with every other number here. It is the single most important figure in this paper for reading the rest of it.

model	build	params	size	accuracy	ROC-AUC	FP/N	p50, 1 thread
majority class (`complete`)	—	—	—	63.08%	0.500	—	—
`smart-turn-v3.0`, zero-shot	int8	8.0M	—	65.64%	0.779	7.08%	—
`smart-turn-v3.2`, zero-shot	int8	8.0M	8.7 MB	70.30%	0.751	13.44%	—
TamilEOT-tiny	fp32	8.0M	32 MB	83.35%	0.904	7.73%	133 ms
TamilEOT-tiny	**int8 dyn**	8.0M	**8.7 MB**	**83.71%**	**0.905**	7.94%	**83 ms**
TamilEOT-tiny	int8 static	8.0M	8.4 MB	79.32%	0.899	4.94%	73 ms
TamilEOT-base	fp32	20.3M	81 MB	86.23%	0.922	8.95%	232 ms
TamilEOT-base	**int8 dyn**	20.3M	21 MB	**86.13%**	**0.921**	9.17%	143 ms
TamilEOT-base	int8 static	20.3M	21 MB	73.46%	0.792	13.70%	125 ms

Table 4: Test split: 4,168 clips from 30 held-out calls, threshold 0.5. FP/N is Smart Turn's convention (see Section 1). Latency is p50, batch 1, one thread, inference only, on an idle Intel i5-12450H; add ∼12 ms for the mel front-end. Released models in bold.

6.2 Main result

Table 4 is the headline. Fine-tuning takes Tamil from 70.30% to **86.13%** — **+15.83 points** — and ROC-AUC from 0.751 to 0.921.

The zero-shot number is worth reading closely, because it is what made the project look tractable before anything was trained. At threshold 0.5, released Smart Turn calls **36.4% of unfinished Tamil turns finished**: an agent using it talks over the user on a third of their mid-sentence pauses. But its ROC-AUC is 0.751, not 0.5. Whisper's encoder already hears Tamil prosody — the *ranking* carries real signal and only the decision boundary is wrong. Threshold tuning alone, swept directly on test, tops out at 74.23%: a ceiling, not a fix.

6.3 Where the gains land

Table 5 breaks accuracy down by provenance, expressed as points above or below a majority-class policy on that bucket.

bucket	tiny	base	
`change`	−7.42	−2.05	speaker change already implies completion
`change_midseg`	−1.34	**+0.45**	first bucket base clears that tiny does not
`hold_intra`	+31.59	**+33.42**	58% of the set — the real EOT problem
`hold_inter`	+31.88	**+35.25**	
`disputed`	−11.76	−6.17	where the two witnesses disagree

Table 5: Accuracy relative to a majority-class policy on each bucket, in points.

The gains are concentrated exactly where they should be: on same-speaker pauses, which is the case a timeout cannot handle and where a majority-class policy is useless.

6.4 What moved the number, and what did not

We measured six training levers (Table 6). One worked.

Capacity, and then the constraint moves. Changing one string, `whisper-tiny` to `whisper-base`, is worth +2.88 accuracy and +0.018 AUC. But the diagnosis does not survive the change. At tiny, the learning curve is flat from 50% to 100% of the data (0.909 → 0.910) — capacity-limited. At base it is still rising (0.922 → 0.931) and the train−dev accuracy gap opens to +11.81 points (98.99% vs 87.18%) — data-limited. *Tiny was capacity-bound; base is data-bound.* Neither diagnostic transfers

lever	measured on	Δ	
whisper-tiny → whisper-base	test accuracy	**+2.88**	the only one that moved
distillation, base → tiny	test accuracy	+0.41	inside the 0.87 spread
funnel relaxation, +19.9% train rows	test accuracy	+0.03	
learning-rate retune	dev AUC	+0.003	reversed once data grew
50% → 100% of data, at tiny	dev AUC	+0.001	flat — capacity-bound
undisputed-only training	test accuracy	−5.11	
class rebalancing	—	—	no problem existed
threshold tuned on dev, base	test accuracy	−1.13	did not transfer
int8 *static* quantisation, base	test accuracy	−12.76	

Table 6: Every training lever measured, ranked. Three separate levers bought about +0.003 AUC each; one bought six times that.

across an architecture change, and re-running the train−dev gap after any capacity change costs one forward pass.

Data, not compute. Because epochs were fixed, a 25%-data run also took a quarter of the optimiser steps, which would confound the learning curve. Re-running at 25% data for 4× the epochs — the same 1,872 steps — returned −0.17 points. Four times the compute on the same data bought nothing; four times the data at fixed compute bought +3.32.

Distillation fails in an instructive way. Distilling base into tiny ($\alpha = 0.3$, $T = 2.0$, loss function the only variable) gives +0.41 accuracy — inside the noise band — and closes 14% of the teacher gap against a textbook 30–60%. Broken down, the entire headline gain is +3.22 on `change`, the bucket where a speaker change already implies completion, while *every bucket that requires hearing prosody got worse*, including `hold_intra` at −0.45. FP/N — the metric that decides whether the agent talks over the user — got worse by 1.41 points. A teacher that memorised its training set has structural regularities left to transfer, and little else.

Class imbalance was not a problem. The set is 1.88:1 complete:incomplete, which looks like something to fix. It is not. `BCEWithLogitsLoss` already carries a per-batch `pos_weight` ≈ 0.531, so the two classes contribute equally; `hold_intra` — 58% of the data and the bucket that decides the score — is already 1.16:1; and the global ratio comes from `change` at 12:1, which is definitional. We record this because the hypothesis was plausible enough to cost a day.

More data, less than expected. Relaxing two funnel gates recovered 2,272 additional train/dev rows — 19.9% more training data — and was worth +0.03 points. The prediction from the learning curve had been +0.5 to +1.2; at tiny's capacity the curve had already saturated. We keep the rows — they cost about a dollar to label — but the honest verdict is that they are not a lever. Notably, relaxing the gate harvested *positives*, not the negatives it was meant to: a short final talk-spurt followed by a pause is usually a short complete answer, not a mid-thought pause.

6.5 The decision threshold

0.5 is inherited from the training loop, not chosen, and neither model peaks there. Tuning it turned out to be harder than it looks.

Picking the dev-set argmax and measuring once on test *lost* base 1.13 points (+0.29 on dev, −1.13 on test). The accuracy-versus-threshold curve is flat near its top and dev is 2,325 clips, so the

argmax is mostly noise: the test sweep's best is 0.65 and the dev sweep's is 0.24, which is the same fact stated twice.

Targeting a *rate* transfers where targeting an argmax does not (Table 7). Holding dev FPR to 10% — the agent interrupts on at most one pause in ten — roughly halves FP/N for about three accuracy points. The direction transfers; the level slips to 12–14% on test. The released default remains 0.5, which is what every number in this paper uses.

operating point	threshold	test accuracy	test FP/N	
tiny, inherited	0.50	83.35%	7.73%	released default
tiny, dev FPR $\leq$ 10%	0.75	80.64%	**5.01%**	
base, inherited	0.50	86.23%	8.95%	released default
base, dev FPR $\leq$ 10%	0.92	82.41%	**4.51%**	

Table 7: Thresholds picked on dev, measured once on test.

6.6 Quantisation

int8 *dynamic* quantisation is free at both sizes: about 3.8× smaller, about 38% faster, and −0.10 to +0.36 accuracy — both inside the noise band. Our tiny build is 8.65 MB against upstream's 8.68 MB, so the released model really is a drop-in replacement as an artefact and not merely as a graph signature.

int8 *static* quantisation is a trap, and the damage scales with capacity. At tiny it costs −4.03 accuracy while AUC barely moves (0.904 → 0.899): the ranking survives and only the decision boundary shifts. At base it costs −12.76 and AUC collapses 0.922 → 0.792, most of the way back to the zero-shot model. 20.3M parameters have a wider activation range than 256 calibration clips can cover.

This is also the clearest case in our results for never reading one metric. tiny int8 static has the best FP/N in Table 4 — 4.94% — and is the second-worst model in it. It was also the variant we expected to win.

7 Serving

7.1 Every accuracy number above scores a clip that production never cuts

Offline evaluation scores a pre-cut window ending exactly 200 ms past the speech offset, because the dataset builder cut it there. Nothing in production cuts a window that way. Live, the window is whatever the VAD hands over, whenever it decides speech stopped.

We measured the difference directly. 500 labelled test boundaries were replayed through the real Silero VAD (`min_silence_duration`= 0.25) and the real streaming adapter in 20 ms frames, and the *same rows* were scored both ways in the same run.

	pre-cut clip	live window
accuracy	86.12%	**83.51%**
false complete (talks over the user)	24	30
FP/N	5.21%	**6.51%**

Serving costs 2.60 points. Paired, the two columns give an identical verdict on 419 of 461 clips (90.9%); McNemar [3] gives $p = 0.09$, so the difference is consistent in direction but not formally significant at $p < 0.05$. It is stable across sample sizes (−2.67 at $n = 187$, −2.60 at $n = 461$). Read it as "about two to three points, probably real, small" — not as a constant.

The cause is measurable and small: the VAD closes at p50 +0.29 s past the labelled offset where the clips stop at +0.20 s. A **90 ms window shift** — 90 ms more trailing silence than training ever showed the model.

7.2 7.8% of boundaries never reach the model

Of the 500 boundaries, the VAD closed on 461 (92.2%); on the other 39 it never closed, so the model was never consulted. The agent framework will not request a prediction below `min_silence_duration`+50 ms. Those cases are not model errors — they are outside the product — but the offline test set counts them, which is one reason offline and live accuracy are not the same quantity. Coverage is 94% on `complete` and 89% on `incomplete`.

This also explains a number that otherwise looks wrong: the pre-cut column reads 86.12% while the full split reads 83.35%. The boundaries a VAD surfaces are the easier ones.

7.3 Latency, with its conditions attached

Model-only latency is in Table 4: 83 ms for tiny int8 at one thread, 143 ms for base. One thread is the number that matters — a server carrying concurrent calls cannot give each one twelve cores — and with all 12 cores those figures fall to 27 ms and 52 ms.

End-to-end through the real adapter on live audio is **120–155 ms**: mel front-end (∼12 ms), ONNX inference, and the thread handoff. These are different spans and both are correct; mixing them is how three wrong latency figures were published in this project before the measurement protocol was fixed. One of them (measured while another job saturated all cores, and 3–5× inflated) produced a wrong shipping recommendation. **A latency number without thread count and machine load attached is not a measurement.**

7.4 What a live call can and cannot tell you

We ran the released model as a complete Tamil voice agent on LiveKit Agents 1.7 and on Pipecat 1.7, with a commercial Indic STT/LLM/TTS stack around it, and recorded instrumented calls.

Against fixed-timeout endpointing on those recordings, the detector cost +120 ms of median endpointing and bought a 35% reduction in utterance fragmentation. We report this as directional only: one speaker, one script, seven utterance blocks per arm.

Three cautions are worth passing on, because each one produced a confident wrong answer first. **(1)** A live call cannot report accuracy. Nobody labelled, per pause, whether that speaker had finished; any accuracy figure from an unlabelled live call is invented. **(2)** The framework's own false-interruption counter read 0 on every arm, because it only fires once the agent has actually started speaking — the commoner damage, a turn committed mid-sentence so the utterance arrives in five pieces, is invisible to it. **(3)** A control arm has to sit on the same commit path. Our first comparison put the baselines on a code path that waits for ASR while the detector arm often had a pre-landed transcript; its apparent win was plumbing, and the run was discarded.

We also caught two harness bugs this way, both of which produced plausible numbers. A replay pump that outran the VAD let the buffer fill past the close point, so the model saw audio a live session would not yet have had; it cost 26 accuracy points and looked exactly like a serving catastrophe. Both bugs were caught by scoring the same rows the known-good way *in the same run*: when the paired column failed to reproduce a known figure, the harness was wrong, not the model. **A live-only number is unfalsifiable.**

8 Limitations

One corpus, one domain. All 116 calls come from a single corpus of narrowband Tamil telephone conversation. We have not measured wideband speech, noisy environments, code-switched Tamil–

English (common in practice), or speakers outside this corpus's demographic. The 30 test calls are unseen, but they are not a different distribution.

The labels are machine-produced and human-validated, not human-produced. Agreement with a human listener is measured at 97.5% on 197 clips and published with its per-class breakdown, but 197 clips is a small validation set and one listener is one listener.

The reproducibility floor is wide. At 0.87 points of run-to-run spread, several results in Table 6 are individually unfalsifiable and are reported as such rather than as findings. A determinism fix exists (`use_deterministic_algorithms`, disabling flash/mem-efficient SDPA) but is untested here and expensive.

Live evidence is thin. The serving replay is properly labelled and paired; the live-call comparison is not, and should be read as an existence proof and a direction, not a benchmark.

Known open items. Regularisation is untried — base reaches 98.99% train accuracy against 87.18% dev, which is heavy memorisation, and SpecAugment, higher dropout and earlier stopping are all unexplored. Short-buffer behaviour is a real mechanism with an unestablished effect: the detector's window is cleared at each turn boundary and right-padded to 8 seconds with zeros, and only 58 of 16,216 training clips are shorter than 8 seconds, so the first prediction of every turn is out of distribution. Three live calls gave +62, +21 and −1 points on the affected rate, which is not a measurement. The correct experiment — truncating test clips to 0.5/1/2/4 s, where labels exist and $n = 4{,}168$ — is not yet run.

9 Conclusion

Tamil semantic end-of-turn detection was not blocked by architecture. Smart Turn v3 is 8M parameters and its training code is public; what was missing was Tamil data with labels anyone had checked. We built that, and the build is the contribution as much as the model is.

Three things generalise past Tamil. First, **validate label rules by listening**: ours were 95.9% right on the class with a recorded witness and 44.4% right on the class without one, and no feature engineering rescued the difference. Second, **establish the run-to-run spread before reporting deltas**: at 0.87 points, most of our levers were unfalsifiable and we would otherwise have reported several of them as wins. Third, **measure what serving costs**: replaying labelled boundaries through the production VAD and adapter cost 2.60 accuracy points and revealed that 7.8% of boundaries never reach the model at all, neither of which is visible offline.

The same pipeline should port to any language with a two-channel conversational corpus, since the label oracle is structural rather than linguistic. That is the next thing we would like someone to do with it.

Availability

Everything below is public and was verified reachable anonymously.

code, pipeline, all experiments	https://github.com/santhosh-005/tamil-eot
models (int8 ONNX)	https://huggingface.co/santhosh-005/smart-turn-tamil
dataset (18,485 boundaries, CC BY 4.0)	https://huggingface.co/datasets/santhosh-005/tamil-eot
inference package	https://pypi.org/project/smart-turn-livekit/

Code is BSD-2-Clause. The dataset derives from SPRING-INX Tamil R1 [5] (CC BY 4.0, SPRING Lab, IIT Madras) and carries the same licence; we do not own the recordings and redistribute them under attribution. The models are fine-tunes of `pipecat-ai/smart-turn` (BSD-2-Clause).

Two pipeline steps — the labeller bake-off and the label-quality analysis — reproduce their reports byte-identically from a bare clone with no corpus, no GPU and no cloud account, using the text-stripped label tables committed to the repository.

Acknowledgements

SPRING Lab, IIT Madras, for releasing SPRING-INX under CC BY 4.0; the Pipecat team for releasing Smart Turn's weights, data and training code, without which this would have been a much larger project.

References

[1] Hyunjong Ok, Suho Yoo, and Jaeho Lee. Speculative End-Turn Detector for Efficient Speech Chatbot Assistant. *arXiv:2503.23439*, 2025.

[2] Erik Ekstedt and Gabriel Skantze. Voice Activity Projection: Self-supervised Learning of Turn-taking Events. In *Interspeech 2022*, pages 5190–5194.

[3] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

[4] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust Speech Recognition via Large-Scale Weak Supervision. In *ICML 2023*, PMLR 202:28492–28518.

[5] Nithya R, Malavika S, Jordan F, Arjun Gangwar, Metilda N J, S Umesh, et al. SPRING-INX: A Multilingual Indian Language Speech Corpus by SPRING Lab, IIT Madras. *arXiv:2310.14654*, 2023.

[6] Silero Team. Silero VAD: pre-trained enterprise-grade voice activity detector. https://github.com/snakers4/silero-vad, 2024.

[7] Pipecat AI. Smart Turn v3: an open-source semantic voice activity detection model. https://github.com/pipecat-ai/smart-turn, 2025.

[8] Thanapol Popit, Natthapath Rungseesiripak, Monthol Charattrakool, and Saksorn Ruangtanusak. Thai Semantic End-of-Turn Detection for Real-Time Voice Agents. *arXiv:2510.04016*, 2025.